

Route Alignment: runA → runB

Phase-2 take-home — technical report (draft)

[Candidate name] • August 31, 2026

*Draft: §1 is complete; §2–§4 are placeholders pending that work. Every number states, where introduced, whether it is **declared** (a label – filename, header field, tool default) or **measured** (computed from file content or capture timestamps); the two are never merged into one number.*

1 Characterising the recordings

runA/runB are two recordings of the same route (cameras `cam0`, `cam5`), delivered as raw HEVC elementary streams (no container – almost no reliable per-file metadata) plus a sidecar `.timestamps.txt` per camera (one capture timestamp per line, ns since epoch, loaded as `int64` throughout – a raw epoch value exceeds `float64`’s exact-integer range). Declared fields came from `ffprobe/ffmpeg` header parsing; measured values came from forcing a full decode and from timestamp-file arithmetic. Code: `src/routealign/{io_video,timestamps}.py`; full JSON per file: `outputs/inspection/`.

Run/cam	Resolution	Size (MB)	Frames (dec./lines)	Decl. fps	Meas. fps	Route (s)	Recorded (UTC)
runA/cam0	1440×1080	129.0	2674 / 2695	10.00	9.992	267.5	2025-02-28 06:24
runA/cam5	1440×1080	121.9	2674 / 2695	10.00	9.992	267.5	2025-02-28 06:24
runB/cam0	1440×1080	70.0	2622 / 2663	10.00	9.989	262.4	2024-04-13 09:12
runB/cam5	1440×1080	64.0	2615 / 2663	10.00	9.989	261.7	2024-04-13 09:12

Table 1: Task 1 summary (full precision in `outputs/inspection/*.json`).

runB was recorded 321 days before runA – two separate real-world sessions. A first-frame check supports this: runA is sunny with hard shadows; runB is overcast with water droplets visible on the camera housing (both cameras), consistent with rain – a qualitative read from individual frames, not a measured statistic.

Frame rate. Three candidate declarations were found and tested, not taken at face value. *Kept*, *10.00 fps*: the HEVC SPS/VUI (`ffmpeg -bsf:v trace_headers`) gives `time_scale/num_units_in_tick=10/1`, independently matching `ffprobe`’s `r_frame_rate`. *Rejected*, *filename token 20*: no field in the file confirms it means fps. *Rejected*, *avg_frame_rate=25/1*: proven, not just argued, to be a content-independent fallback – a synthetic clip encoded at a known true 17 fps still reports `avg_frame_rate=25/1`. The **measured** rate, 9.992/9.989 fps, comes from none of these: it is $(N_{\text{decoded}} - 1)/\text{span}$, span from the timestamp file. It is close to the declared value, but that closeness is a finding, not an assumption.

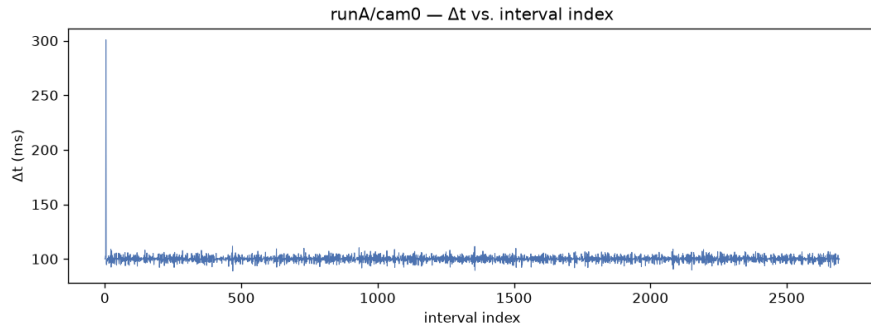


Figure 1: Interval between consecutive frames vs. index, **runA/cam0** (all four files show the same shape). A single 300.8 ms spike (runA) / 396.9 ms (runB) sits in the first few frames; the two frames bracketing each gap are visually indistinguishable, ruling out a scene event – most likely a one-off recording start-up cost, mechanism unconfirmed. No other file shows any comparable gap.

Frame-count discrepancy. Every file has more timestamp lines than frames that actually decode (runA: 2695 vs. 2674, both cameras; runB: 2663 vs. 2622/2615 – *not* identical between cameras, so the usable range must be tracked per camera). All four files are an exact multiple of 262144 B (256 KiB), consistent with – not proof of – a writer that stopped without a final flush; no truncation warning appears on decode either way, and no byte-level NAL scan has confirmed an incomplete final unit. Consequence for Task 2: the mapping’s H0 convention (timestamp line k = decoded frame k) holds only up to each file’s decoded count; lines beyond it have no video and are marked as such, never forced into a match.

2 Matching frames between runA and runB

Placeholder – Task 2, 40% of the grade. Written once the alignment pipeline (descriptors, DTW, verification, confidence, ground-truth evaluation) is implemented and measured against labelled anchors: representation, search and the monotonicity constraint, correct vs. confident-wrong, outputs/mapping.csv, an honest accuracy estimate with its ground truth, and failure cases.

3 Keep/discard policy

Placeholder – Task 3, written once §2 is final (the policy depends on its output): which frames to keep or discard and why, and what is attached to each surviving frame (timing confidence, correspondence status, condition, split assignment) so a downstream consumer cannot silently misuse it.

4 Reproducibility and the proved-vs-assumed ledger

*Placeholder: every non-obvious claim in this report tagged proved / measured-with-caveats / assumed (H0 chief among them), mirroring **plan.md**’s ledger, plus exact commands and pinned versions.*